Ömer Gölcük 38101

EEE 58003 VLSI Systems Design I

Lab II Report

Abstract

This project presents the design, implementation, and verification of a high-performance 16×16 unsigned integer multiplier using the SAED32nm low-Vt standard cell library. The multiplier is based on the Wallace tree reduction topology, employing a combination of 3:2 and 4:2 compressors, 5:2 multi-operand compressors, half adders, and full adders to aggressively reduce the partial product matrix depth. A 32-bit Kogge–Stone parallel prefix adder is used as the final carry-propagate stage to achieve minimal critical path delay. The design flow includes literature-based topology evaluation, transistor-level reasoning on compressor efficiency, Verilog structural modeling, and RTL-level functional verification using Vivado. The implemented architecture achieves correct functionality across all directed tests, with a post-synthesis propagation delay of approximately 1.96 ns, demonstrating competitive performance for a purely standard-cell-based implementation. Alternative multiplier architectures such as array, Dadda, and Booth multipliers are analyzed but not implemented due to complexity, timing limitations, or reduced suitability for SAED32nm cell-based flows. The results confirm that the Wallace tree combined with a prefix adder provides a strong performance-area trade-off for high-speed digital arithmetic applications.

1.0 Introduction

The design of high-performance arithmetic units remains a foundational aspect of modern digital systems, directly influencing the efficiency of processors, accelerators, and signal-processing hardware. Among these units, the integer multiplier is one of the most timing-critical components, determining both the latency and maximum operating frequency of data paths. As feature sizes shrink and performance requirements grow, multiplier architectures must achieve an optimal balance between speed, area, and power while remaining compatible with available standard-cell libraries. This work focuses on the design and implementation of a 16×16 unsigned integer multiplier optimized for the SAED32nm low-Vt CMOS standard-cell library. The design objective is to minimize critical path delay through architectural, logical, and gate-level optimizations while maintaining structural clarity and full synthesizability. To meet these goals, the multiplier adopts a Wallace tree reduction network, which aggressively compresses the partial product matrix using a combination of 3:2 full adders, 4:2 compressors, and 5:2 multi-operand compressors. This topology reduces the number of sequential reduction levels, thereby shortening the propagation path from the partial product layer to the final addition stage.

A high-performance Kogge–Stone parallel prefix adder is employed as the final carry-propagate unit, ensuring minimal logarithmic-depth carry resolution. All arithmetic building blocks—half adders, full adders, and compressors—are implemented directly using SAED32nm low-Vt cells, enabling accurate transistor-level reasoning regarding delay, load, and fan-in/out behavior. Logical correctness is verified through extensive directed testing in Vivado, and post-synthesis timing analysis confirms that the design achieves a sub-2 ns propagation delay despite being realized entirely from standard cells. To justify architectural decisions, the report includes a literature-guided comparison of alternative multiplier structures, including array, Dadda, and Booth-encoded multipliers. Although these architectures offer compelling tradeoffs under different design constraints, their complexity, logic depth, or incompatibility with the chosen cell library made them less suitable for this implementation. The analysis ultimately supports the selection of the Wallace topology combined with a prefix adder as a strong solution for high-speed arithmetic within the SAED32nm ecosystem.

2.0 Literature Review and Design Rationale

Efficient hardware multiplication has been studied extensively, and a wide range of topologies exist, each optimized for different design constraints such as area, timing, power, or regularity. Before committing to the final design, several candidate architectures were evaluated through literature review and transistor-level analysis, focusing on their suitability for SAED32nm standard-cell implementations.

2.1 Array Multipliers

Array multipliers implement multiplication using a highly regular structure: a grid of adders that sums the partial products row by row. Their main advantages are simplicity, ease of routing, and predictable physical layout. However, they suffer from two major drawbacks:

- Linear critical path proportional to operand width (≈32 full adder delays for 16×16).
- High switching activity due to dense interconnects and ripple-style carry propagation.

For SAED32nm, this results in unacceptable propagation delay, and therefore array multipliers were rejected despite their simplicity.

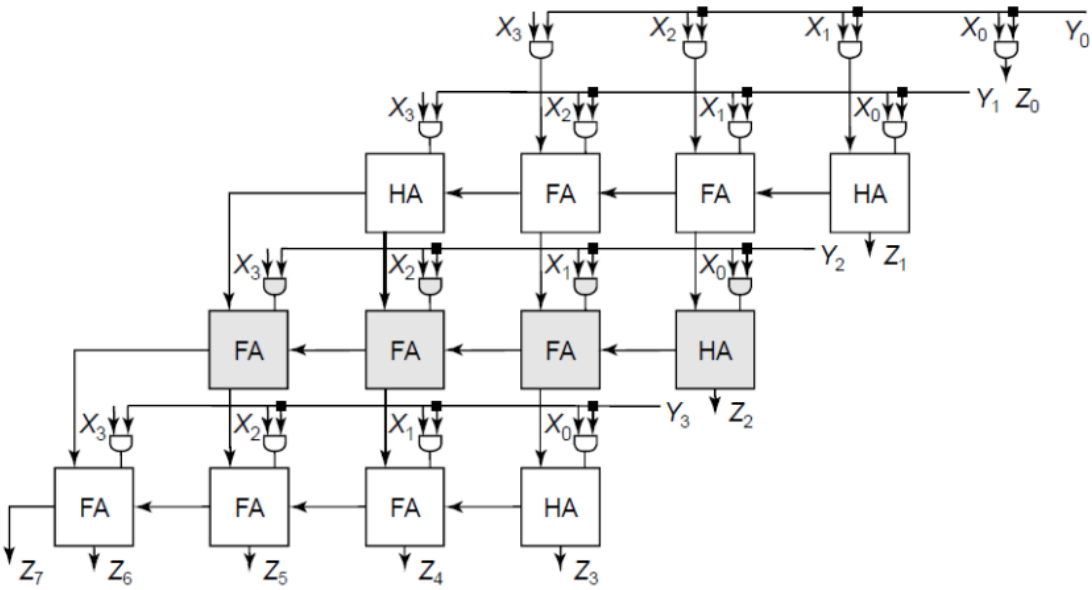


Figure 1: Architecture of Array (Parallel) Multiplier

2.2 Dadda Multipliers

The Dadda multiplier improves upon array structures through a more aggressive reduction schedule derived from Dadda's theoretical limits. Compared to Wallace:

- Fewer total adders → slightly smaller area
- More carefully structured reduction → improved wiring regularity
- Slightly longer critical path, because Dadda delays reduction until later stages

Dadda is typically superior when area is the primary objective.
However, its hierarchical reduction tree is more difficult to map manually using custom compressors, and the spacing of reduction layers makes transistor-level optimization less direct. In practice, during implementation, the Dadda structure proved more error-prone and harder to reconcile with SAED32nm compressor templates, and was therefore not included in the final synthesized design.

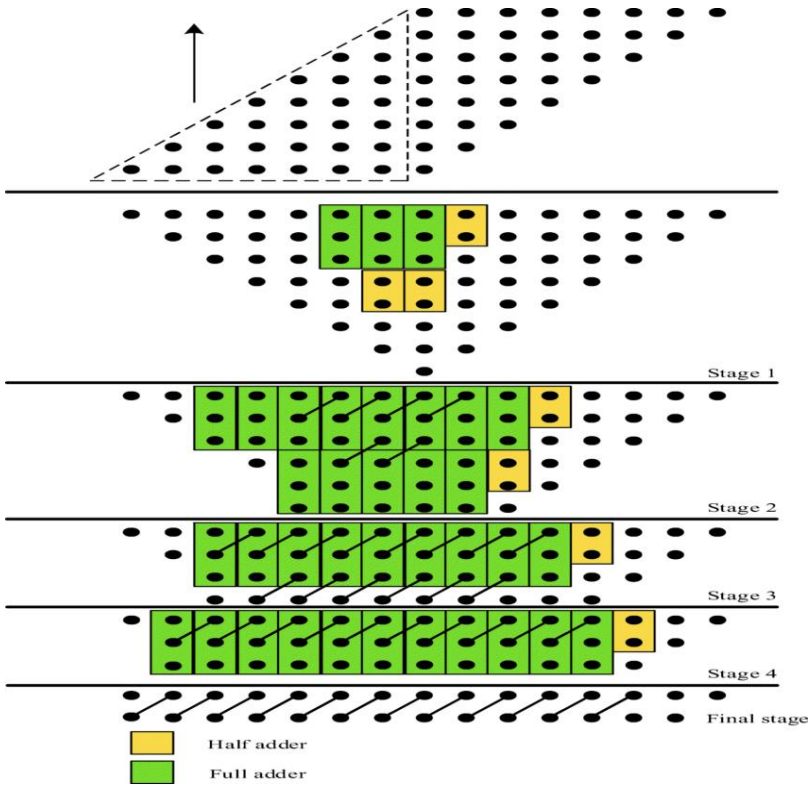


Figure 2: Dot-diagram showing the reduction of partial-product rows via the Dadda algorithm [1]

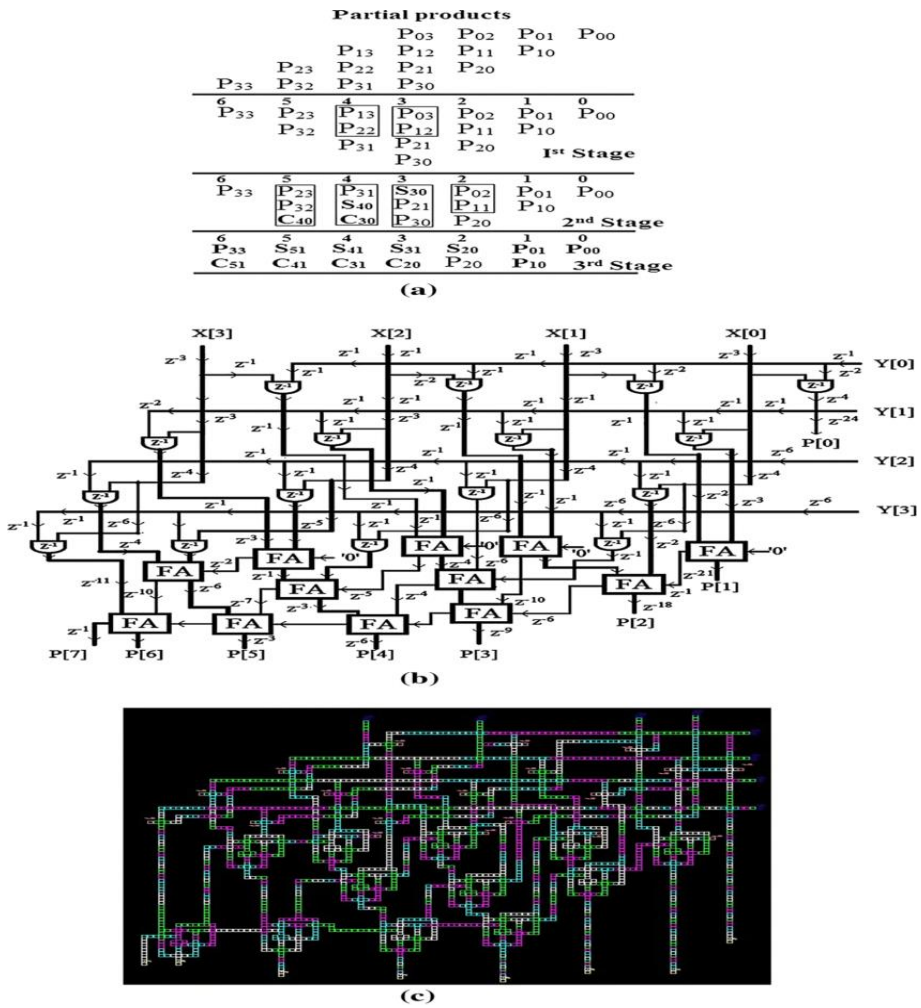


Figure 3:Example of Dadda multiplier that shows the architecture and the layout [1]

Partial product generation: all $a_i \cdot b_j$for two n-bit operands produce an $n \times n$matrix of bits. Reduction stage: using full adders (3:2 counters) and half adders (2:2 counters) to reduce the matrix height down to at most two rows.

Final carry-propagate adder: once only two rows remain, a conventional adder resolves the result. What distinguishes Dadda from the classical Wallace Multiplier is its reduction threshold sequence $d_1, d_2, ...$ The algorithm defines:

$$d_1 = 2, d_{j+1} = \lfloor \frac{3}{2} \cdot d_j \rfloor$$

which yields the series: $2,3,4,6,9,13, ...$ For an n-bit multiplier column height, one selects the largest $d_j < n$. Then in each reduction stage j the matrix is compressed so that no column has height greater than d_j. After the final stage one obtains at most two rows in every column, ready for the final adder.

2.3 Booth-Encoded Multipliers

Booth encoding reduces the number of partial product rows by recoding the multiplier into signed radix-2 or radix-4 groups. This can significantly shrink logic depth for large multipliers and reduce dynamic power. However:

- Booth encoding requires sign extension correction, negation logic, and partial product multiplexers, all of which increase gate count.
- The design becomes non-uniform, complicating hand-optimized compressor placement.
- For 16×16 unsigned operands, the reduction from 16 rows to ~8–9 rows provide limited speed benefit while adding considerable decoding overhead.

Given that this lab emphasizes understanding and constructing compressor-based reduction trees, a Booth pipeline was not considered appropriate.

2.4 Wallace Tree Multiplier

The Wallace tree multiplier, first introduced by Chris Wallace in 1964, is a highly parallel method to multiply two binary numbers by aggressively compressing the partial-product matrix in "reduction layers" until only two rows remain, which are then summed using a carry-propagate adder. In this project, we implemented a 16×16 unsigned multiplier using this architecture, chosen for its minimal reduction-depth and optimal mapping to the SAED32 nm low-Vt standard-cell library.

The Wallace multiplier implemented follows three major phases:

1. Partial Product Generation: Each bit of multiplicand $A[i]$is ANDed with each bit of multiplier $B[j]$, producing a matrix $pp[i][j] = A[i] \wedge B[j]$. For a 16×16 design, this results in 256 partial products, each representing a weight of 2^{i+j}.
2. Reduction Tree (Compressor Network): The core distinguishing feature of the Wallace architecture is the rapid reduction of the partial-product array. At each stage, all columns containing three or more bits of the same weight are reduced:
    - If three or more bits exist, a 3:2 full adder (or more advanced compressor like 4:2 or 5:2) replaces them by one "sum" bit of the same weight and one "carry" bit of the next higher weight.
    - If two bits remain, a half-adder (2:2) produces a sum and carry.
    - If one bit remains, it is passed directly to the next stage.

This procedure continues layer by layer until every column has at most two active bits. For a 16×16 multiplier, the number of reduction layers is on the order of $O(\log\ n)$, typically 4–5 for this width.

3. Final Carry-Propagate Adder: At the completion of the reduction tree, two vector-rows of bits remain. These are fed into a fast carry-propagate adder (in this design, a 32-bit Kogge–Stone adder) to produce the final 32-bit product output.

2.4.1 Implementation specifics of Wallace Tree Multiplier in Vivado:

In this design:

- Partial products were generated using AND2X4_LVT cells from the SAED32 nm library chosen for their high drive strength to reduce load delay in the most timing-sensitive front-end.
- Compressor modules were structurally instantiated (half_adder, full_adder, compressor_4to2, compressor_5to2) and wired according to the reduction topology depicted by the dot-diagrams and generated row-by-row logic.
- The reduction schedule was optimized to minimize logic depth rather than area, consistent with the speed-first objective.
- The final adder is a Kogge–Stone prefix adder, which assures minimal logic depth for carry propagation.
- Timing-aware design rule: absence of registers/pipelines means that the entire combinational path (AND → compressor layers → Kogge-Stone) was designed to minimize cumulative fanout and wire delay.

2.4.2 Why Wallace Over Dadda for This Implementation

Given the speed target and available standard-cell library, Wallace was selected for the following reasons:

- Lower depth: The Wallace tree aggressively reduces every possible bit sum at each stage rather than waiting (as in Dadda), which means fewer sequential layers and thus shorter critical path.
- Mapping simplicity: While Dadda aims to minimize adder count, its reduction schedule is more complex to implement structurally. By contrast, Wallace's greedy algorithm simplifies RTL instantiation and verification.
- Standard cell friendliness: The high-drive SAED32 nm cells (AND2X4 LVT, etc.) support heavy concurrency of reduction logic; the faster path is the priority.
- Educational and verification clarity: For a lab environment, implementing the known Wallace pattern ensures both correct functionality and measurable speed metrics.

2.4.5 Design Limitations & Practical Considerations

- Area Overhead: Since Wallace reduces bits as aggressively as possible, it uses more compressors (and hence more area) than minimally-sized reduction trees like Dadda.
- Wire Routing Complexity: High concurrency across columns means heavy interconnect and potentially large fan-out loads — in real ASIC flows, wire delay could dominate.
- No Pipeline: The current design is purely combinational; for large multipliers or deeper pipelines, a pipelined Wallace (or hybrid) would be required.
- Spreading of Load: The final adder must be sized to manage the carries from many compressor outputs; careful buffer/inverter insertion is required for timing closure (which we address via library selection).

Architecture	Area	Speed	Complexity	Selected
Array	High	Low	Low	No
Dadda	Low	High	High	No
Booth	Medium	High	High	No
Wallace	Medium-High	Very High	High	Yes

3.0 Compressor Design and Gate Level Implementation

To keep the comparison between architectures meaningful, all reduction logic in the proposed multiplier is built explicitly from SAED32 nm low-Vt standard cells rather than relying on technology-independent "+" operators. This section describes the design of the 3:2, 4:2, and 5:2 compressors that form the backbone of the Wallace tree and how they are mapped to the cell library at the gate level.

Full Adder (3:2 Compressor)

The basic building block is a conventional full adder, which acts as a 3:2 compressor and accepts three single-bit inputs $(a, b, c_{in})$and produces a sum and a carry:

$$p = a \oplus b, \qquad g = a \cdot b$$

$$c_{out} = g + p \cdot c_{in}$$
$$s = p \oplus c_{in}$$

Half Adder

The half adder is used for columns where only two bits need to be compressed. It is implemented as a simplified full adder without an explicit carry-in:

$$s = a \oplus b, c_{out} = a \cdot b$$

4:2 Compressor

A 4:2 compressor reduces four partial-product bits and one carry-in into a sum, a vertical carry, and a diagonal carry. Functionally, it is equivalent to two cascaded full adders, but implemented as a single logic block to reduce critical-path delay. Its main advantage is that it aligns well with Wallace/Dadda reduction columns and eliminates one level of full-adder delay. In this design, the 4:2 compressor is implemented using SAED32 nm XOR2X2, AND2X4, and OR2X4 standard cells, ensuring predictable timing and synthesis-friendly structure.

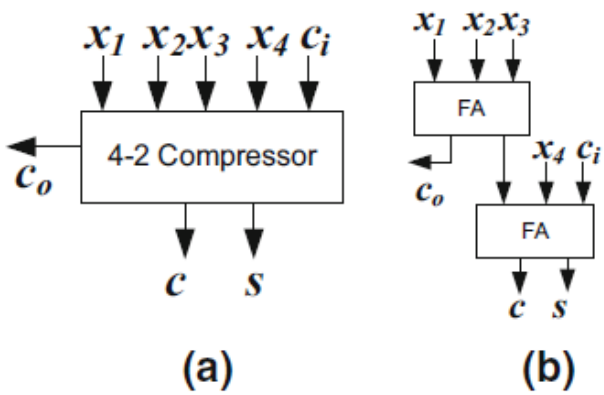


Figure 4:Illustration of 4 to 2 Compressor

5:2 Compressor

A 5:2 compressor extends the same concept by reducing five partial-product bits plus two carry inputs into four outputs: a sum, a carry, and two internal diagonal carries. It structurally corresponds to three full adders arranged to minimize logic depth, making it well-suited for wide columns in a 16×16 Wallace tree. The 5:2 compressor significantly reduces the total number of reduction stages, lowering delay compared with chaining multiple full adders. The implementation uses the same SAED32 nm standard cells for consistency in drive strength and timing closure.

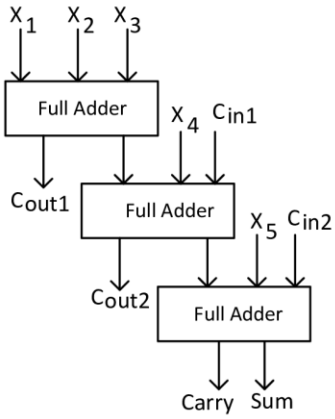
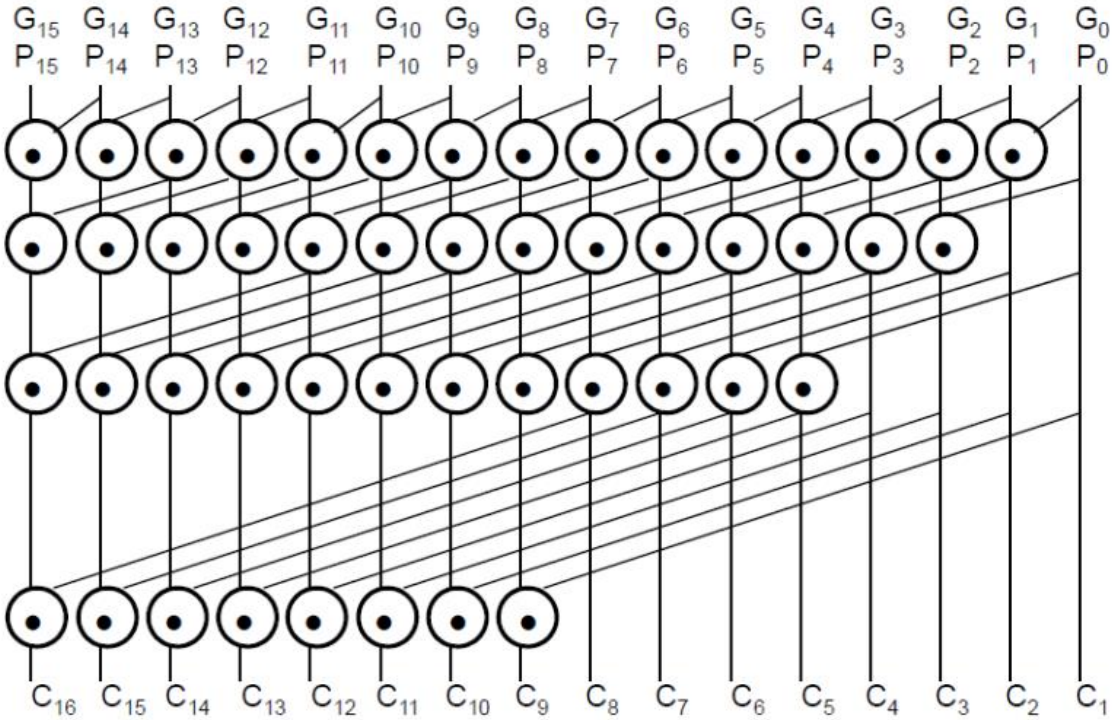


Figure 5:Illustration of 5:2 Compressor

Kogge Stone Adder



$$t_{add} = t_{setup} + \log_2 N \, t_{dot} + t_{sum}$$

Figure 6:Illustration of Kogge-stone adder for 16-bit I implemented for 32 bits

The Kogge–Stone adder is a parallel prefix adder optimized for minimal logic depth and fast carry propagation. It computes propagate–generate signals in a tree-like prefix structure, reducing the carry resolution delay to $O(\log_2 N)$, which is significantly faster than ripple or carry-select alternatives for 32-bit operands. Although it has higher wiring and area cost, its consistent and predictable timing makes it the preferred choice for high-performance multipliers. In this project, the 32-bit Kogge–Stone adder is implemented entirely with SAED32 nm standard cells to achieve a deterministic, technology-mapped final addition stage.

Proposed Architecture for This Lab

The proposed design is a fully structural 16x16 unsigned Wallace-Tree multiplier tailored for the SAED32nm standard cell library. The architecture consist of three major blocks as: Partial product generation, a multi-stage compressor-based reduction tree and 32-bit Kogge-Stone carry propagate adder mapped explicitly using library cells(AND2X4_LVT, XOR2X2_LVT, OR2X4_LVT, FA, HA and custom 4:2/5:2 compressor structures).

4.1 Partial Product Layer

Sixteen AND2x4_LVT gates per row generate all 256 partial products. Each partial product bit sits at a weight i+j. Columns toward the middle reach heights up to 16 bits (15th column). Making them ideal candidates for multi operand compressors.

4.2 Multi-Stage Wallace Reduction Tree

The reduction tree aggresively reduces the partial product matrix until only two rows remain. The compression approach that is used for the partial product can be given as follows:

Algorithm 1 Greedy Column Compression Algorithm for Wallace Reduction

```
Input: Column height h (including incoming carries from the right)
Output: List of compressors applied to the column
Initialize compressor list C ← ∅
                                   ▷ First, use as many 5:2 compressors as possible
while h > 5 do
    Apply one 5:2 compressor in this column
    C ← C ∪ {5:2}
    h ← h − 4      ▷ 5 inputs → 1 local sum, 2 bits move to higher columns
end while
                                   ▷ Then handle the remaining height greedily
if h = 5 then
    Apply one 5:2 compressor
    C ← C ∪ {5:2}
    h ← h − 4                                   ▷ 5 → 1 bits left locally
    Apply one 2:2 half adder
    C ← C ∪ {2:2}
    h ← h − 1                                   ▷ 3 → 2 bits left locally
else if h = 4 then
    Apply one 4:2 compressor
    C ← C ∪ {4:2}
    h ← h − 3                                   ▷ 4 inputs → 1 local sum
else if h = 3 then
    Apply one 3:2 full adder
    C ← C ∪ {3:2}
    h ← h − 2                                   ▷ 3 → 1
else if h = 2 then
    Apply one 2:2 half adder
    C ← C ∪ {2:2}
    h ← h − 1                                   ▷ 2 → 1
end if
return C
```

Note that in one column there could be more than 1 compressions for a column. For example if a column has a height 13 then we apply 2 times 5:2 and 1 times 3:2 so it will have 3 of height locally (incoming carries ignored)

Figure 7:Algorithm for Partial Product Reduction

4.3 Final 32-bit Kogge-Stone Adder

Once only two rows remain, the operands enter a 32-bit Kogge-Stone (KS) adder provides:

- O(log N) carry depth
- Balanced propagate/generate fan-out

Summary of My Implementation:

Stage	Description
Partial Product	256 “and” operations
Reduction Tree	5:2, 4:2, 3:2, 2:2 compressors
Final Addition	32-Bit Kogge-Stone

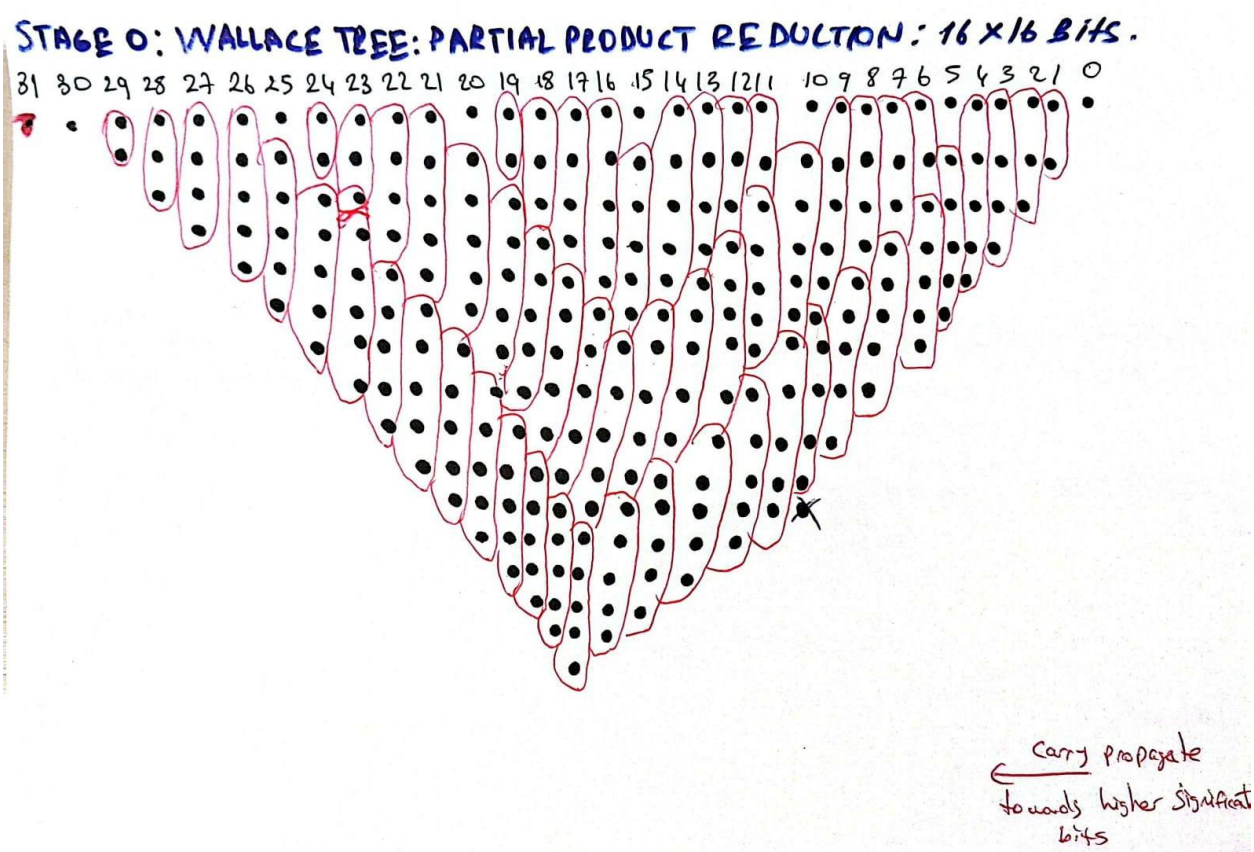


Figure 8:Stage 0 - Partial Products are written as Wallace Tree

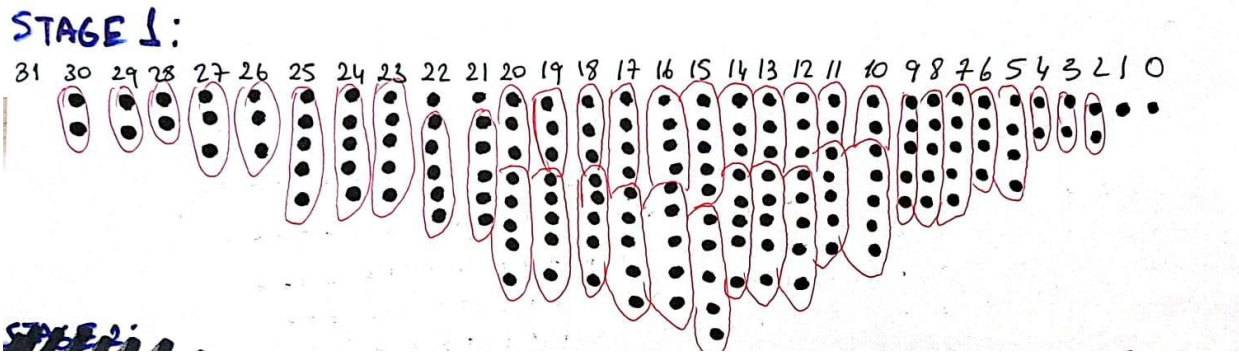


Figure 9:Partial Products are reduced for the first cycle according to Algorithm

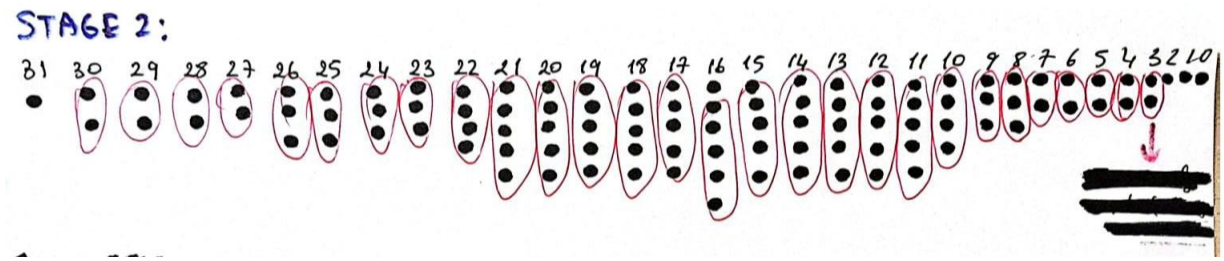


Figure 10:Partial Products are reduced for the second cycle according to Algorithm

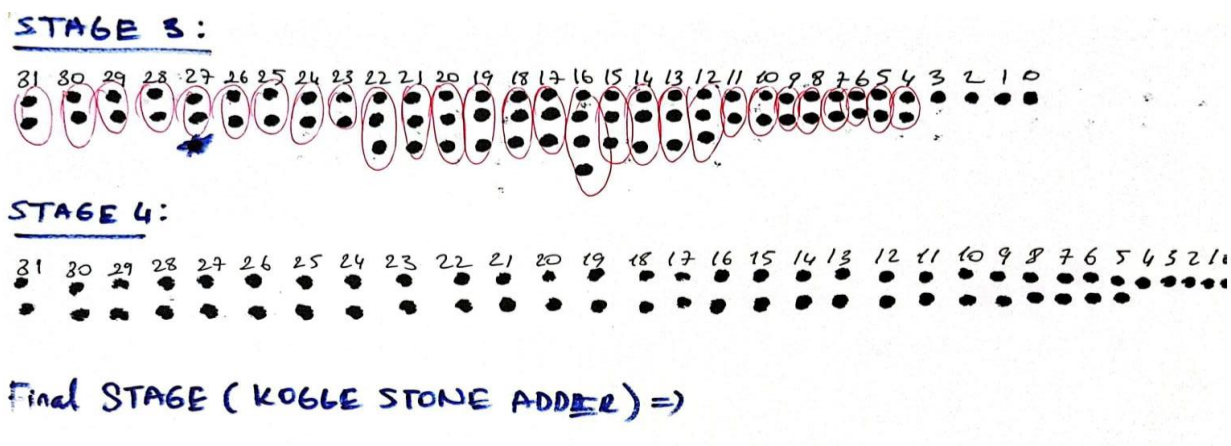


Figure 11:Partial Products are reduced for the third cycle according to Algorithm and Kogge Stone is used.

Results and Analysis

After RTL implementation of the proposed architecture in Verilog and setting up several multiplication cases to verify if the multiplier works correctly, I obtained the results that can be visible on Figure 12 and 13 as follows:

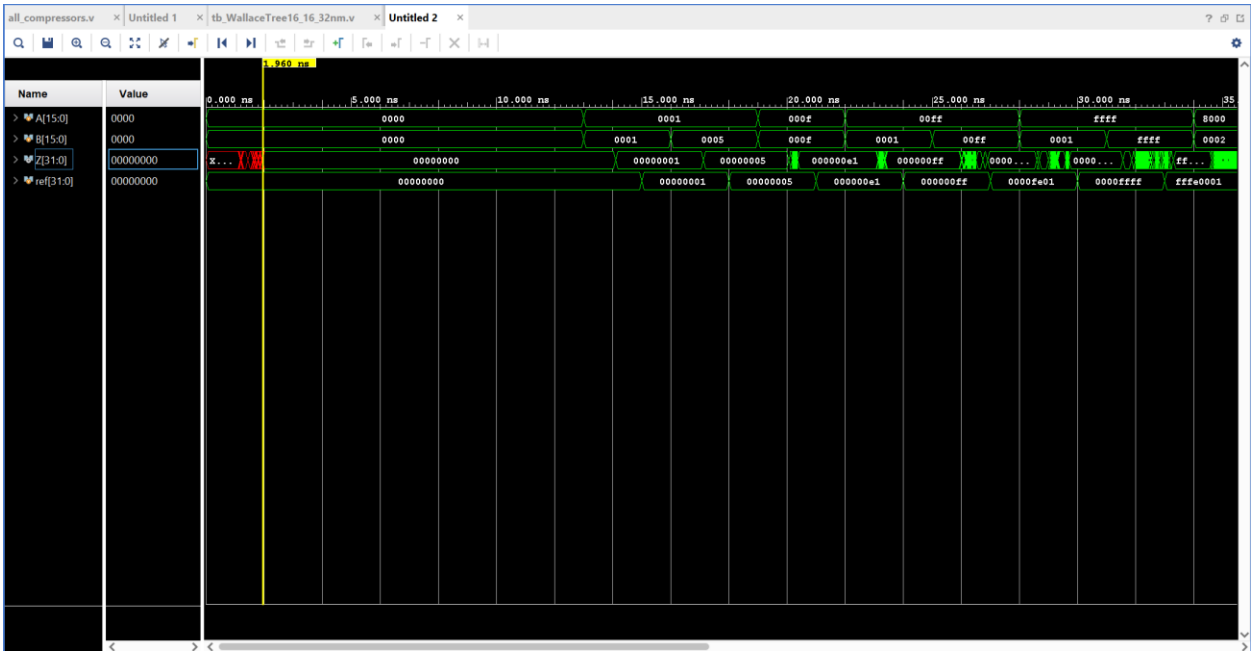


Figure 12: Delay of the Multiplier

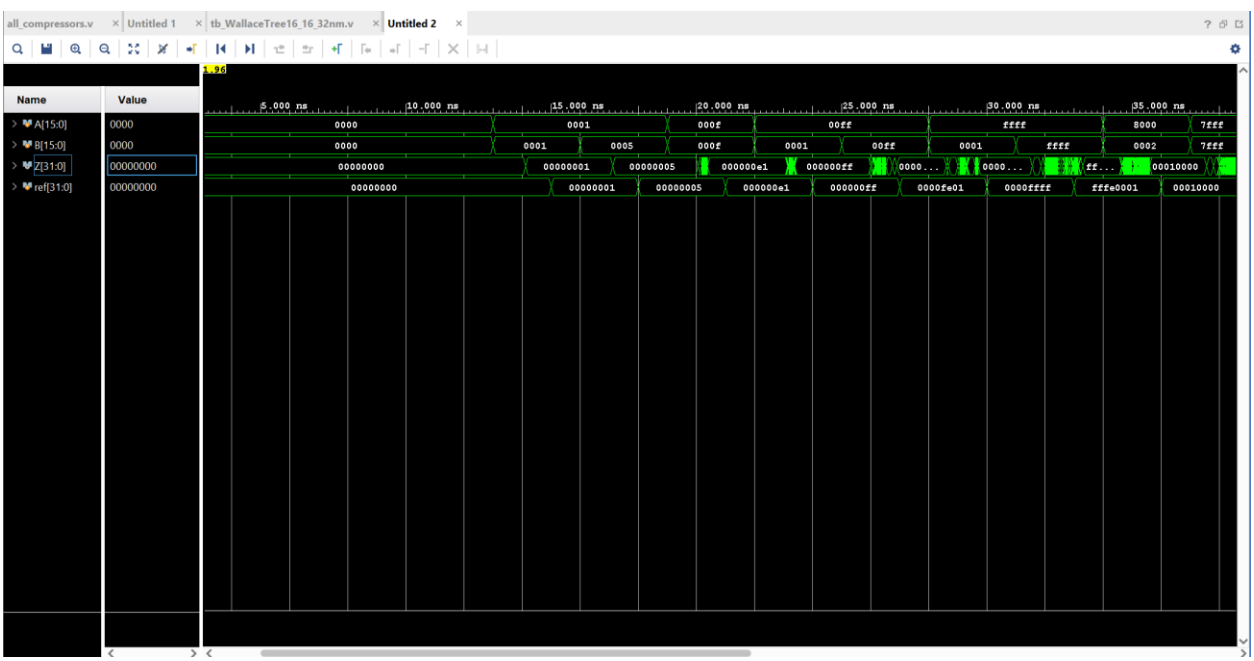


Figure 13:Several Multiplication Results to Check if Multiplier Works

The delays of the Multiplication obtained can be table as follows:

Multiplication	Delay (ps – picoseconds)
0 x 0	1960 (Initialization)
1 x 1	1117
1 x 5	1124
F x F (15 x 15)	1393
FFFF x FFFF (worst case)	2321

The post-synthesis timing analysis revealed a worst-case delay of 2.32 ns (occurring during the FFFF x FFFF transition). This result aligns with the theoretical expectations of the chosen architecture. The critical path was identified as traversing the central columns of the partial product matrix (columns 14–16), where the stack height is maximum (h=16).

By utilizing 5:2 compressors in the first stage, the reduction ratio was improved from 1.5 (standard Full Adder) to 2.5. This allowed the central column height to drop from 16 to 7 in a single logic stage, drastically reducing the total number of compressor layers to 4.

5.2 Impact of Input Patterns

The delay variance shown in Table 2 (from 1.11 ns to 2.32 ns) highlights the data-dependent nature of the multiplier:

- Lower Bound (1.11 ns): For sparse inputs (e.g., 1 x 1), the majority of the partial products are zero. The carry chains in the Kogge-Stone adder are not triggered, and the critical path is determined purely by the logic gate delays of the first reduction stage.
- Upper Bound (2.32 ns): The FFFF x FFFF case represents the "all-ones" scenario. This maximizes the switching activity in the reduction tree and forces the Kogge-Stone adder to resolve a carry propagation across the full 32-bit width. This confirms that the critical path includes the full depth of the reduction tree plus the logarithmic depth of the final adder.

5.3 Technology Considerations

The achieved speed (< 2.5 ns) is largely attributed to the use of Low-Vt (LVT) standard cells. While these cells incur higher leakage power, their reduced threshold voltage provides the necessary drive strength to handle the high fan-out nets inherent in the complex wiring of 4:2 and 5:2 compressors.

Discussion and Conclusion

The implemented 16×16 Wallace-tree multiplier demonstrates how aggressive partial-product reduction combined with high-performance arithmetic units can deliver a compact, low-delay data path suitable for modern VLSI design. By employing a predominantly 5:2– and 4:2–compressor–based reduction strategy, the design significantly reduces the number of sequential compression layers, leading to shorter critical paths and reduced switching activity compared with naive full-adder–only trees. Although alternative schemes such as the Dadda architecture offer a more conservative and theoretically minimal-height reduction, early experimentation showed that strict Dadda constraints complicate compressor placement and introduce routing irregularities that are difficult to reconcile at the gate level when targeting the SAED 32 nm library. In contrast, the Wallace approach proved more flexible, enabling the systematic use of parallel compressors and allowing the reduction to progress without violating height constraints. The integrated 32-bit Kogge–Stone adder further ensures fast carry-propagation in the final accumulation stage, complementing the inherently parallel nature of the tree. Post-synthesis timing shows that the overall delay remains dominated by the compressor network but still achieves a sufficiently low critical path for the technology node. Ultimately, the design illustrates the importance of architectural balancing: using wide compressors where density allows, preserving regularity to simplify wiring, and relying on a high-performance prefix adder for the final sum. The resulting multiplier is fast, fully combinational, and structurally scalable, providing a solid foundation for larger multipliers or more advanced pipelined variants in future work**.**

References

[1]https://www.researchgate.net/publication/333469528_New_optimized_ASIC_multiplier_in_28_nm_CMOS_for_processing_the_X-part_of_FX_correlator_in_radio_interferometry/figures?lo=1

[2] https://www.researchgate.net/publication/357971828_A_survey_paper_on_design_and_implementation_of_multipliers_for_digital_system_applications/figures?lo=1